⚡ JavaScript Promises Cheat Sheet

A concise reference for working with Promises for asynchronous JavaScript.

> **Tip**
>
> Complete Dev Roadmap with SaaS Boilerplate at thedevspace.io/

Creating Promises

```
// create a promise that wraps fetch
const p = new Promise((resolve, reject) => {
  fetch("/api")
    .then((res) => resolve(res))
    .catch((err) => reject(err));
});
```

- Promises represent eventual completion or failure of async operations
- Use `resolve` for success and `reject` for errors
- Prefer higher level APIs like `fetch` or `async` / `await` over manual promise construction when possible

Chaining & Error Handling

```
// chain promises and handle errors
doAsync()
  .then((result) => doMore(result))
  .catch((err) => console.error(err));
```

- Chain `.then()` for sequential async steps
- Use `.catch()` to handle errors from the whole chain
- Returning a value from `.then()` passes it to the next handler
- For clearer flow prefer async/await with try/catch

Parallel Patterns: all, race, any, allSettled

```
// parallel promise helpers
Promise.all([p1, p2]).then(([a, b]) => {});
Promise.race([p1, p2]).then((first) => {});
Promise.any([p1, p2]).then((firstFulfilled) => {});
Promise.allSettled([p1, p2]).then((results) => {});
```

- `Promise.all` waits for all to resolve and rejects on first error
- `Promise.race` settles with the first settled promise
- `Promise.any` resolves with the first fulfilled promise and ignores rejections
- `Promise.allSettled` returns every outcome without short circuiting

Converting Callbacks & Timeouts

```
// wrap a callback style API in a promise
function wait(ms) {
  return new Promise((resolve) =>
setTimeout(resolve, ms));
}
```

- Wrap callback APIs in promises for easier composition
- Use timeouts and AbortController to avoid hanging requests
- Keep promises pure for memoization and testing when possible

Best Practices

```
// example safe consumption with async/await
async function safeFetch(url) {
  try {
    const r = await fetch(url);
    return await r.json();
  } catch (e) {
    console.error(e);
    return null;
  }
}
```

- Avoid deep nesting by returning promises or using async/await
- Handle errors centrally and provide timeouts where appropriate
- Prefer AbortController for cancellable fetches
- Document expected promise shapes and rejection behavior